

El algoritmo de Shor

Factorización de enteros en tiempo polinómico cuántico

Computación Cuántica — Ingeniería Informática — ITBA

Documento fuente en Markdown. Contiene el desarrollo matemático completo, los diagramas de circuito y las notas de orador por diapositiva. La versión visual (Canva) se deriva de este archivo.

Idioma: español neutro. Nivel asumido: compuertas de 1 y varios qubits, notación de Dirac, circuitos cuánticos. No se reexplica qué es un qubit o una superposición.

Slide 1 — Portada

El algoritmo de Shor

Cómo una computadora cuántica factoriza enteros y por qué eso rompe RSA

Materia: Computación Cuántica · ITBA

Expositores: [Nombre 1] · [Nombre 2]

Slide 2 — Hoja de ruta

De qué vamos a hablar

1. Motivación y por qué factorizar rompe RSA
2. El problema: factorización como *order-finding* (hallar el período)
3. Prerrequisitos sobre el número N
4. El algoritmo: parte clásica + parte cuántica
5. Por qué el período permite factorizar (la cadena algebraica completa)
6. La QFT y el circuito cuántico
7. Ejemplo resuelto con $N = 15$: un fallo ($k = 14$) y un éxito ($k = 7$)
8. Complejidad: Shor vs. el mejor clásico (GNFS)
9. Implicancias, estado del hardware y criptografía post-cuántica

Slide 3 — Motivación: por qué nos importa

- Gran parte de la seguridad de Internet (TLS, firmas, certificados) se apoya en que **factorizar un entero grande es difícil** para una computadora clásica.
- "Difícil" significa: el mejor algoritmo clásico conocido (GNFS) tarda **tiempo sub-exponencial** — para una clave RSA de 2048 bits, fuera del alcance práctico.
- En 1994 Peter Shor mostró un algoritmo **cuántico** que factoriza en **tiempo polinómico**. Eso convierte un problema "imposible" en uno "tratable".
- Consecuencia directa: si existiera una computadora cuántica suficientemente grande y confiable, **RSA y los esquemas basados en logaritmo discreto (DH, DSA, ECDSA) dejarían de ser seguros**.

Slide 4 — Repaso express de RSA

RSA ya se vio en la cursada de Criptografía (Clase 04). Acá solo lo necesario para entender por qué factorizar N lo rompe.

Generación de claves (textbook RSA):

- Se eligen dos primos p, q y se define el módulo $N = p \cdot q$.
- Se calcula $\phi(N) = (p - 1)(q - 1)$.
- Se elige e con $\gcd(e, \phi(N)) = 1$, y se calcula d tal que $e \cdot d \equiv 1 \pmod{\phi(N)}$.
- Clave pública: (N, e) · Clave privada: (N, d) .

Operaciones:

- Cifrado: $c \equiv m^e \pmod{N}$
- Descifrado: $m \equiv c^d \pmod{N}$

Lo público es (N, e) . El secreto efectivo es d , y d se deriva de $\phi(N)$.

Slide 5 — Por qué factorizar N rompe RSA

La cadena es directa:

conocer $p, q \rightarrow \phi(N) = (p-1)(q-1) \rightarrow d \equiv e^{-1} \pmod{\phi(N)} \rightarrow$ clave privada

- El atacante conoce N y e (son públicos).
- Si logra **factorizar** $N = p \cdot q$, calcula $\phi(N)$ trivialmente.
- Con $\phi(N)$ y e , obtiene d con el **algoritmo de Euclides extendido** (eficiente).
- Con d descifra cualquier mensaje y firma como si fuera el dueño de la clave.

La seguridad de RSA se apoya por completo en que nadie pueda factorizar N .

Shor ataca ese único punto.

Slide 6 — El problema real: factorización $\equiv$ order-finding

Shor **no** factoriza por fuerza bruta. Reduce la factorización a otro problema: **hallar el período (orden) de una función.**

Dado N a factorizar y un k coprimo con N , se define:

$$f(x) = k^x \bmod N$$

Esta función es **periódica**: existe un mínimo $r > 0$ tal que

$$f(x + r) = f(x) \quad \text{para todo } x \quad \Leftrightarrow \quad k^r \equiv 1 \pmod{N}$$

A ese r se lo llama el **orden de k módulo N** (o período de f).

Idea central: conocer r permite, con álgebra elemental + GCD, recuperar factores de N . Y hallar r es lo único que necesita la computadora cuántica.

Slide 7 — Prerrequisitos sobre el input N

Antes de correr Shor, se descartan clásicamente los casos triviales. **N debe cumplir:**

1. **N no es primo.** Si fuera primo, no hay nada que factorizar (test de primalidad clásico y eficiente, p. ej. Miller-Rabin).
2. **N no es par.** Si lo fuera, 2 ya es un factor. Se chequea mirando el último bit.
3. **N no es una potencia perfecta** n^x (con n, x enteros, $x \geq 2$). Si lo fuera, la raíz n es un factor, y se halla con métodos clásicos eficientes (probar raíces x -ésimas para x hasta $\log_2 N$).

Si N pasa estos filtros, es un **compuesto impar con al menos dos factores primos distintos** — el caso para el que Shor está diseñado.

Slide 8 — El corazón: el período de $f(x) = k^x \bmod N$

$$f(x) = k^x \bmod N$$

- Es **periódica** con período r = orden de k módulo N .
- Hallar r es el único paso que requiere la computadora cuántica.
- ¿Por qué no clásico? Clásicamente habría que evaluar f en muchísimos puntos para detectar la repetición: para N de miles de bits, el período puede ser astronómicamente grande. **No se conoce un método clásico eficiente para hallar r .**
- La computadora cuántica evalúa f en **superposición sobre todos los x a la vez** y luego usa interferencia (la QFT) para extraer la periodicidad.

Todos los demás pasos del algoritmo (GCD, chequeos, derivar factores) son **clásicos y polinómicos**. La ventaja cuántica vive enteramente acá.

Slide 9 — Esqueleto del algoritmo: clásico vs. cuántico

CLÁSICO

0. Filtrar N (no primo, no par, no potencia perfecta)
1. Elegir k al azar, $1 < k < N$
2. $g = \gcd(k, N)$ (Euclides)
 - └ si $g \neq 1 \rightarrow$ ¡suerte! g es factor. FIN.
 - └ si $g = 1 \rightarrow$ seguir

CUÁNTICO

3. Hallar el período r de
 $f(x) = k^x \bmod N$
(superposición + QFT)

CLÁSICO

4. Si r es IMPAR $\rightarrow$ descartar, volver a 1 (otro k)
5. Calcular $p = k^{(r/2)} \bmod N$
6. Si $p + 1 \equiv 0 \pmod{N}$, es decir $p = N-1$
 $\rightarrow$ descartar, volver a 1 (otro k)
7. Factores: $\gcd(p - 1, N)$ y $\gcd(p + 1, N)$ (Euclides)

Slide 10 — La parte clásica, en detalle

1. Elegir k uniformemente al azar en $1 < k < N$.
2. GCD inicial: $g = \gcd(k, N)$.
 - Si $g \neq 1$: g es un factor no trivial $\rightarrow$ listo (caso afortunado, sin cuántica).
 - Si $g = 1$: k es coprimo con $N \rightarrow$ tiene sentido buscar su orden.
3. (cuántico: obtener r)
4. Paridad: si r es impar $\rightarrow$ no sirve, elegir otro k .
5. Calcular $p = k^{(r/2)} \bmod N$.
6. Chequeo de fallo: si $p = N - 1$ (equivale a $k^{(r/2)} \equiv -1$) $\rightarrow$ no sirve, otro k .
7. Extraer factores: $\gcd(p - 1, N)$ y $\gcd(p + 1, N)$ dan factores no triviales.

Todos estos pasos son **clásicos y eficientes** (exponenciación modular rápida + Euclides).

Slide 11 — GCD y Euclides: honestidad sobre su uso

¿Cómo se calcula $\text{gcd}(a, N)$? Con el algoritmo de Euclides:

```
gcd(a, b):  
    while b ≠ 0:  
        a, b = b, a mod b  
    return a
```

Idea: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, reduciendo el problema hasta que un término sea 0.

¿Se usa de verdad o es solo didáctico?

- **Sí se usa en la práctica.** Euclides es eficiente (tiempo polinómico, $O((\log N)^2)$ o mejor) y es exactamente lo que se ejecuta para los GCD al implementar Shor en, por ejemplo, Qiskit.
- Lo que **no** se hace clásicamente es el **paso de hallar el período r** : ese es el único que no tiene método clásico eficiente conocido y por eso va a la computadora cuántica.

Moraleja: los GCD y el resto del post-procesamiento son baratos. La dificultad está concentrada, otra vez, en hallar el período.

Slide 12 — Por qué el período permite factorizar (1/5): el punto de partida

Partimos del hecho que define a r :

$$r \text{ es el período de } f(x) = k^x \bmod N \iff k^r \equiv 1 \pmod{N}$$

Reescribiendo:

$$k^r \equiv 1 \pmod{N} \iff k^r - 1 \equiv 0 \pmod{N} \iff N \mid (k^r - 1)$$

Es decir: N divide a $k^r - 1$.

A partir de acá vamos a mostrar, encadenadas, cuatro cosas:

- (a) por qué r debe ser **par**,
- (b) por qué exigimos $k^{(r/2)} + 1 \not\equiv 0 \pmod{N}$,
- (c) por qué $\gcd(k^{(r/2)} \pm 1, N)$ da factores,
- (d) por qué todo esto funciona con **alta probabilidad**.

Slide 13 — (a) Por qué r debe ser par

Tenemos $N \mid (k^r - 1)$. Si r es par, podemos definir $p = r/2$ entero y factorizar como diferencia de cuadrados:

$$k^r - 1 = (k^{(r/2)})^2 - 1 = (k^{(r/2)} - 1)(k^{(r/2)} + 1)$$

Esta factorización es **la que abre la puerta**: N divide al producto de dos números más chicos y manejables, $k^{(r/2)} - 1$ y $k^{(r/2)} + 1$.

Si r es impar:

- $r/2$ no es entero, **no existe** esa factorización en enteros,
- no hay forma de partir $k^r - 1$ en dos factores tratables,
- **se descarta este k y se reintenta** con otro.

Conclusión: necesitamos r par para tener los dos "ladrillos" $k^{(r/2)} \mp 1$.

Slide 14 — (b) Por qué se exige $k^{(r/2)} + 1 \not\equiv 0 \pmod{N}$

De (a): $N \mid (k^{(r/2)} - 1)(k^{(r/2)} + 1)$.

Para que esto sea **útil**, N tiene que **repartir** sus factores primos entre los dos paréntesis (un primo de N en uno, el resto en el otro). El intento **falla** si N divide **enteramente a uno solo** de los factores:

- Si $N \mid (k^{(r/2)} - 1) \rightarrow k^{(r/2)} \equiv 1 \pmod{N}$. Pero eso significaría que $r/2$ ya es un período, **contradiciendo que r es el período mínimo**. Así que este caso no puede ocurrir si r es el orden real.
- Si $N \mid (k^{(r/2)} + 1) \rightarrow k^{(r/2)} \equiv -1 \pmod{N}$, es decir $k^{(r/2)} \bmod N = N - 1$.
Este sí puede ocurrir, y es el que se chequea en las diapositivas como $p + 1 = N$ (con p el residuo). **Es el modo de fallo del "caso feo" $k = 14$.**

En ambos fallos, el GCD del paso siguiente daría algo **trivial** (1 o N). Por eso, si $p = N - 1$, se descarta k y se reintenta.

Slide 15 — (c) Por qué $\gcd(k^{(r/2)} \pm 1, N)$ da factores no triviales

Superado el chequeo (b), sabemos que N **no divide por completo a ninguno** de los dos factores $k^{(r/2)} - 1$ ni $k^{(r/2)} + 1$. Pero sí divide al **producto**.

Como $N = p_1 \cdot p_2 \cdot \dots$ (sus primos), y ningún paréntesis se lleva todos los primos:

- algún primo de N divide a $k^{(r/2)} - 1$,
- y otro primo de N divide a $k^{(r/2)} + 1$.

Por lo tanto:

$$\gcd(k^{(r/2)} - 1, N) \quad \text{y} \quad \gcd(k^{(r/2)} + 1, N)$$

comparten cada uno parte de los primos de N , y devuelven **divisores propios** (ni 1 ni N).

Para $N = 15$ esto dará exactamente **3 y 5**.

Es el **GCD** —no la resta directa— lo que aísla el factor. Y como Euclides es clásico y eficiente, **este último paso es clásico y barato**.

Slide 16 — (d) "Alta probabilidad": la cota cuantitativa

¿Con qué probabilidad un k random (coprimo con N) cumple **ambas** condiciones — r par y $k^{(r/2)} \equiv -1 \pmod{N}$?

Teorema (Nielsen & Chuang, *Quantum Computation and Quantum Information*, Teorema A4.13 / §5.3.1):

Sea N un compuesto impar con m factores primos distintos. Si se elige k uniformemente al azar entre los coprimos con N , entonces

$$\Pr[r \text{ par y } k^{(r/2)} \equiv -1 \pmod{N}] \geq 1 - 1 / 2^{(m-1)}$$

- Como N (tras los filtros) tiene $m \geq 2$ primos distintos: **probabilidad de éxito** $\geq 1 - 1/2 = 1/2$ por intento.
- Si N tiene más primos distintos, la cota mejora (más cerca de 1).
- Con éxito $\geq 1/2$ por intento, **el número esperado de reintentos es** ≤ 2 . Pocos k bastan.

Nota de fuentes: el paper de la cátedra (Ugwuishi et al., 2020) describe el procedimiento pero **no da esta cota cuantitativa**; se cita por eso la fuente estándar (Nielsen & Chuang). Shor (1994) prueba un resultado equivalente.

Slide 17 — La parte cuántica: hallar el período

Objetivo del subprograma cuántico: dado k y N , hallar r tal que $k^r \equiv 1 \pmod{N}$.

Se usan **dos registros**:

- **Registro de conteo** (counting / "input"): t qubits, inicializado en $|0\rangle$.
Va a contener la información de fase de la que se extrae r .
Se elige t con $2^t \geq N^2$ (margen para que las fracciones continuas separen bien).
- **Registro de trabajo** (work): $n = \lceil \log_2 N \rceil$ qubits, inicializado en $|1\rangle$.
Va a contener los valores $k^x \bmod N$.

Pasos (esquema, detalle en las próximas slides):

1. Superposición uniforme en el registro de conteo (Hadamards).
2. Exponenciación modular controlada: $|x\rangle|1\rangle \rightarrow |x\rangle|k^x \bmod N\rangle$.
3. **QFT inversa** sobre el registro de conteo.
4. **Medir** el registro de conteo $\rightarrow$ estimación de s/r .
5. **Fracciones continuas** (clásico) $\rightarrow$ recuperar r .

Slide 18 — La QFT: qué es

La Transformada de Fourier Cuántica sobre n qubits ($M = 2^n$ estados base) actúa así:

$$\text{QFT } |j\rangle = (1/\sqrt{M}) \sum_{k=0}^{M-1} e^{2\pi i \cdot j \cdot k / M} |k\rangle$$

- Mapea un estado base $|j\rangle$ a una **superposición uniforme con fases que giran** a una "frecuencia" proporcional a j .
- Sobre una superposición periódica (como la que produce la exponenciación modular), las fases **interfieren constructivamente** en los múltiplos de M/r y **destructivamente** en el resto.
- Resultado: al medir, los valores probables están concentrados cerca de $c \approx s \cdot M / r \rightarrow$ de ahí sale r .

Implementación eficiente (forma producto): se construye con Hadamards y rotaciones de fase controladas R_d (fase $e^{2\pi i/2^d}$):

$$R_d = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^d} \end{bmatrix}$$

Usa $O(n^2)$ compuertas.

Slide 19 — QFT vs. DFT/FFT clásica

	DFT / FFT (clásica)	QFT (cuántica)
Sobre qué actúa	Vector de M amplitudes explícitas en memoria	Amplitudes de una superposición de $n = \log_2 M$ qubits
Costo	FFT: $O(M \log M) = O(2^n \cdot n)$	$O(n^2) = O((\log M)^2)$ compuertas
Salida	Todas las M componentes, legibles	Estado transformado; al medir, una sola muestra (probabilística)
Acceso a resultados	Lectura completa del espectro	No se puede leer todo: la medición colapsa

La diferencia clave:

- La QFT es **exponencialmente más rápida** en cantidad de operaciones...
- ...pero **no entrega el espectro completo**: solo se puede *muestrear* una salida.
- El arte de Shor es diseñar el circuito para que **lo que se muestrea codifique justo el período** r .

Slide 20 — Estimación de fase: el motor del order-finding

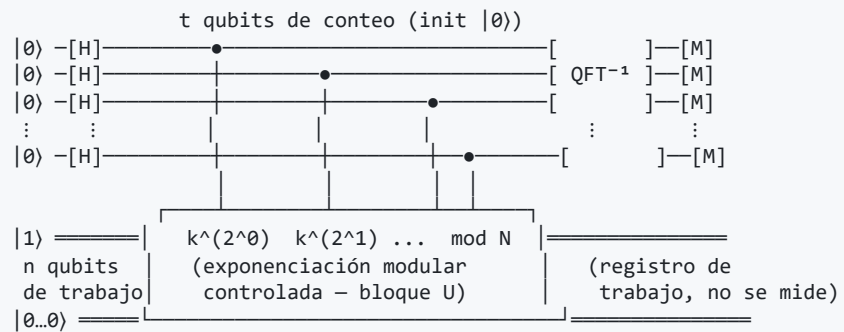
El subprograma de período es una **estimación de fase** del operador U :

$$U |y\rangle = |k \cdot y \bmod N\rangle$$

- Los autovectores de U tienen autovalores $e^{2\pi i s/r}$ (con $s = 0, \dots, r-1$).
- El estado inicial $|1\rangle$ del registro de trabajo es una **superposición de esos autovectores**.
- Aplicando U controlada 2^j veces desde cada qubit de conteo, se "imprime" la fase s/r en el registro de conteo.
- La **QFT inversa** convierte esa fase en un número medible $c \approx s \cdot 2^t / r$.

$k^x \bmod N$
estados base } periodicidad $r \rightarrow$ fases $s/r \rightarrow \text{QFT}^{-1} \rightarrow \text{medir } c \approx s \cdot 2^t / r$

Slide 21 — El circuito cuántico completo (diagrama)



Lectura del circuito:

1. H en los t qubits de conteo $\rightarrow$ superposición uniforme de todos los x .
2. Bloque de **exponenciación modular controlada**: cada qubit de conteo j controla $U^{(2^j)}$, dejando $|x\rangle |k^x \text{ mod } N\rangle$.
3. QFT^{-1} sobre el registro de conteo.
4. **Medición** del registro de conteo $\rightarrow c$. (El registro de trabajo no se mide.)

Slide 22 — El bloque costoso: exponenciación modular controlada

$U : |y\rangle \rightarrow |k \cdot y \bmod N\rangle$ y , encadenando: $|x\rangle|1\rangle \rightarrow |x\rangle|k^x \bmod N\rangle$

- Se trata como un **bloque abstracto** en el circuito (no desarrollamos su construcción interna a nivel compuertas).
- Internamente: sumadores modulares, multiplicadores modulares y exponenciación por cuadrados controlada.
- **Es el bloque más costoso del algoritmo**: domina el conteo de compuertas y de qubits auxiliares (ancillas). La complejidad total de Shor está gobernada por este paso.

En implementaciones reales (Qiskit y similares) esta es la parte que más recursos consume y la que más se optimiza. La QFT, en comparación, es barata.

Slide 23 — De la medición al período: fracciones continuas

La medición da un entero c con $c / 2^t \approx s / r$ (para algún s desconocido).

Para recuperar r se usa el **desarrollo en fracciones continuas** de $c / 2^t$:

sus **convergentes** dan la mejor aproximación racional con denominador acotado, y ese denominador es (un divisor de) r .

Método (clásico, eficiente):

$$c/2^t = a_0 + 1/(a_1 + 1/(a_2 + \dots)) \rightarrow \text{convergentes } p_i/q_i \rightarrow r = q_i \text{ (el adecuado)}$$

Importante para $N = 15$: todos los k coprimos tienen $r \in \{2, 4\}$, que son **potencias de 2**. Como el registro tiene 2^t estados, las fracciones s/r salen **exactas** ($s/2$, $s/4$) y las fracciones continuas son triviales. **No hay "caso feo" de fracciones continuas sucias con $N = 15$.**

Un ejemplo donde las fracciones continuas hacen trabajo no trivial requeriría un N mayor (p. ej. $N = 21$, con períodos como $r = 6$). No lo fabricamos acá: con $N = 15$ el post-procesamiento es exacto por construcción.

Slide 24 — Ejemplo $N = 15$: preparación

Objetivo: factorizar $N = 15$.

Filtros (prerrequisitos):

- ¿Primo? No ($15 = 3 \cdot 5$).
- ¿Par? No (impar).
- ¿Potencia perfecta n^x ? No.
- $\rightarrow 15$ es compuesto impar con 2 primos distintos. **Apto para Shor.**

Tamaño de registros (versión de libro):

- Trabajo: $n = \lceil \log_2 15 \rceil = 4$ qubits.
- Conteo: $2^t \geq 15^2 = 225 \rightarrow t = 8$ qubits ($2^8 = 256$).

Notación (¡atención!): en lo que sigue, $p = k^{(r/2)} \bmod 15$ es un **residuo**, no un número primo. Los factores buscados son 3 y 5.

Vamos a ver **dos casos** con el mismo $N = 15$:

- **Caso feo (fallo):** $k = 14$.
- **Caso feliz (éxito):** $k = 7$.

Slide 25 — Caso feo (fallo): $k = 14$

1. Coprimidad: $\gcd(14, 15) = 1$ ✓ (coprimo, seguimos).

2. Período (parte cuántica): $f(x) = 14^x \bmod 15$:

x:	0	1	2	3	4	...
14^x :	1	14	1	14	1	...

→ $r = 2$ (par ✓)

3. r es par → calculamos $p = 14^{(r/2)} \bmod 15 = 14^1 \bmod 15 = 14$.

4. Chequeo de fallo $p + 1 = N$:

$p + 1 = 14 + 1 = 15 = N$ → ¡FALLA!

Esto es $14^{(r/2)} \equiv -1 \pmod{15}$: N divide por completo a $(p + 1)$.

Si igual calculáramos los GCD, saldrían **triviales**:

$\gcd(p - 1, N) = \gcd(13, 15) = 1$ $\gcd(p + 1, N) = \gcd(15, 15) = 15$

→ ni 1 ni 15 sirven como factor.

Conclusión: $k = 14$ **no factoriza**. Se descarta y se reintenta con otro k .

Slide 26 — Caso feliz: $k = 7$ — parte clásica

1. Coprimidad: $\gcd(7, 15) = 1$ ✓.

2. Período (parte cuántica, lo resolvemos en detalle en las próximas slides):

$$f(x) = 7^x \bmod 15 \quad \rightarrow \quad r = 4$$

3. $r = 4$ es par ✓ $\rightarrow p = 7^{(r/2)} \bmod 15 = 7^2 \bmod 15 = 49 \bmod 15 = 4$.

4. Chequeo $p + 1 = N$?:

$$p + 1 = 4 + 1 = 5 \neq 15 \quad \rightarrow \quad \text{PASA } \checkmark \quad (7^2 \equiv -1 \bmod 15)$$

5. Extraer factores con GCD:

$$\begin{aligned} \gcd(p - 1, N) &= \gcd(4 - 1, 15) = \gcd(3, 15) = 3 \\ \gcd(p + 1, N) &= \gcd(4 + 1, 15) = \gcd(5, 15) = 5 \end{aligned}$$

$\rightarrow$ Factores de 15: 3 y 5. ✓

Slide 27 — Caso $k = 7$ — parte cuántica: la periodicidad

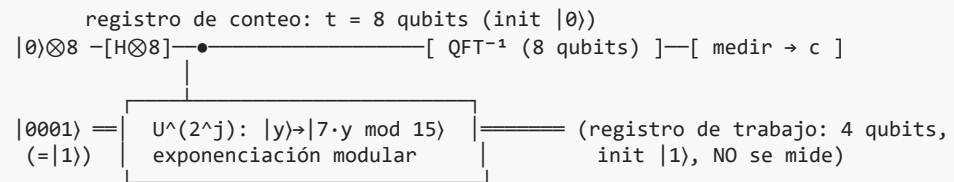
$f(x) = 7^x \bmod 15$, evaluada en superposición por el circuito:

x :	0	1	2	3	4	5	6	7	...
7^x :	1	7	4	13	1	7	4	13	...
									
	período $r = 4$								

El registro de trabajo, tras la exponenciación modular, queda **entrelazado** con el de conteo: para cada x , guarda $7^x \bmod 15 \in \{1, 7, 4, 13\}$.

La secuencia **1, 7, 4, 13, 1, 7, 4, 13, ...** se repite cada 4 pasos → esa es la periodicidad que la QFT^{-1} va a convertir en algo medible.

Slide 28 — Caso $k = 7$ — el circuito concreto



- Conteo: 8 qubits ($2^8 = 256 \geq 15^2 = 225$).
- Trabajo: 4 qubits, inicializado en $|1\rangle = |0001\rangle$.
- Bloque U : multiplicación por 7 módulo 15, aplicada 2^j veces controlada por el qubit j de conteo.
- QFT^{-1} sobre los 8 qubits de conteo, luego medición.

Slide 29 — Caso $k = 7$ — mediciones y fracciones continuas

Con $r = 4$ y $2^t = 256$, la medición del registro de conteo arroja, con alta probabilidad, uno de los valores $c \approx s \cdot 256 / 4 = s \cdot 64$ ($s = 0, 1, 2, 3$):

$c \in \{0, 64, 128, 192\}$

(picos de la QFT^{-1})

Para cada c se calcula $c / 256$ y se aplican fracciones continuas:

c	c / 256	fracción reducida	denominador	¿da $r = 4$?
0	0/256	0	—	inútil ($s=0$) → repetir
64	64/256	1/4	4	✓ $r = 4$
128	128/256	1/2	2	candidato $r=2$; $7^2=49 \equiv 4 \neq 1$ → falla, repetir
192	192/256	3/4	4	✓ $r = 4$

- $c = 64$ y $c = 192$ dan directamente $r = 4$.
- $c = 128$ reduce a $1/2$ (porque $s = 2$ y $r = 4$ comparten el factor 2): da el candidato $r = 2$, que se verifica ($7^2 \bmod 15 = 4 \neq 1$) y se descarta → se repite la medición.
- $c = 0$ no aporta ($s = 0$) → se repite.

Con repetir la medición unas pocas veces, se obtiene $r = 4$ con altísima probabilidad.

Slide 30 — Caso $k = 7$ — cierre del ejemplo

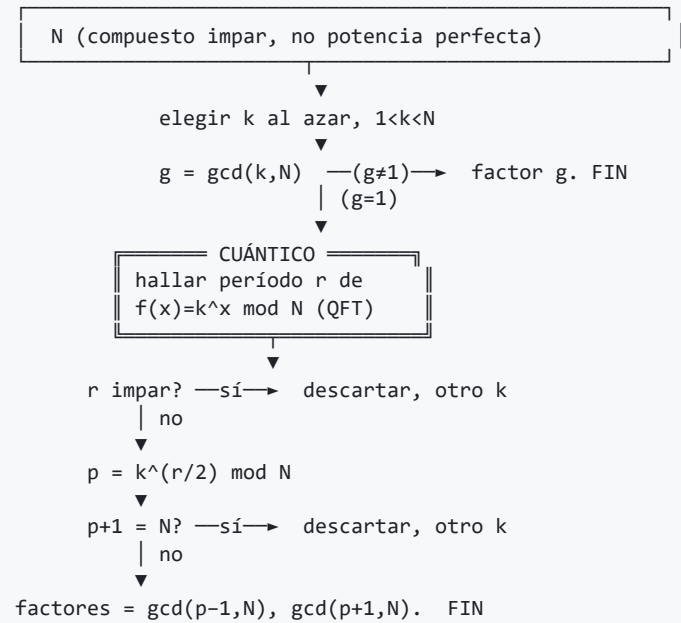
Recapitulación del flujo completo con $k = 7$:

```
N = 15 → filtros OK → k = 7 → gcd(7,15)=1
└─[CUÁNTICO]→ período r = 4 (medición 64/256 → 1/4)
→ r par ✓
→ p = 7^2 mod 15 = 4
→ p+1 = 5 ≠ 15 ✓ (no es el modo de fallo)
→ gcd(4-1,15) = 3      gcd(4+1,15) = 5
→ 15 = 3 × 5    ✓✓
```

Lo que mostró el ejemplo:

- El único paso cuántico fue hallar $r = 4$.
- Todo lo demás (filtros, GCD, chequeos, álgebra) fue **clásico y trivial**.
- Comparado con $k = 14$, se ve que Shor es **probabilístico**: el k importa.

Slide 31 — El flujo completo, de un vistazo



Slide 32 — Complejidad: Shor vs. el mejor clásico (GNFS)

Comparamos **tiempo** (no espacio) para factorizar un entero de $n = \log_2 N$ bits.

Mejor clásico conocido — General Number Field Sieve (GNFS): sub-exponencial

$$\exp\left(\left(\frac{64}{9} \right)^{1/3} + o(1) \right) \cdot (\ln N)^{1/3} \cdot (\ln \ln N)^{2/3}$$

Crece más lento que exponencial puro, pero **mucho más rápido que cualquier polinomio**.

Shor (cuántico): polinómico

$$\begin{aligned} &\tilde{O}(\log^2 N) \text{ en compuertas cuánticas (con multiplicación rápida)} \\ &\approx O(\log^3 N) \text{ en la versión estándar} \end{aligned}$$

más post-procesamiento clásico polinómico (Euclides, fracciones continuas).

Salto cualitativo: **sub-exponencial** → **polinómico**. No es "una constante mejor": es otra clase de complejidad.

Slide 33 — Crecimiento comparativo (intuición)



- Para números chicos, la diferencia no se nota.
- Para **claves RSA reales (2048+ bits)**, GNFS se va a tiempos astronómicos mientras Shor se mantiene tratable (en una máquina cuántica ideal suficientemente grande).
- Esa brecha es exactamente la amenaza criptográfica.

Slide 34 — Implicancias en la criptografía actual

Qué rompe Shor (si hay hardware suficiente):

- **RSA** (factorización) — cifrado y firma.
- **Diffie-Hellman, ElGamal, DSA** (logaritmo discreto) — variante de Shor.
- **ECDH, ECDSA** (logaritmo discreto en curvas elípticas) — también caen, e incluso con *menos* qubits que RSA del mismo nivel de seguridad.

Qué NO rompe (o solo debilita levemente):

- **Criptografía simétrica (AES)**: Shor no aplica. El ataque cuántico relevante es **Grover**, que solo da una aceleración cuadrática → se mitiga **duplicando el tamaño de clave** (AES-128 → AES-256).
- **Funciones de hash (SHA-2/3)**: igual, Grover cuadrático → usar hashes más largos.

Resumen: la **criptografía de clave pública clásica es la víctima principal**; la simétrica sobrevive con claves más largas.

Slide 35 — Estado actual del hardware cuántico (2025–2026)

- **Google "Willow"** (fines 2024): 105 qubits superconductores; demostró **corrección de errores "por debajo del umbral"** (el error lógico baja al agregar qubits físicos). Hito cualitativo, no una máquina factorizadora.
- **IBM — hoja de ruta a tolerancia a fallos:** sistema **"Starling"** para 2029 con **~200 qubits lógicos** y 100 millones de operaciones corregidas; hitos intermedios *Loon* (2025), *Kookaburra* (2026), *Cockatoo* (2027). Meta de **~1000 qubits lógicos** a inicios de la década de 2030, usando códigos **qLDPC**.
- **Realidad para Shor:** todavía **no existe** una máquina capaz de correr Shor sobre números criptográficamente relevantes. Falta tolerancia a fallos a escala.

Distinción clave: **qubits físicos** (ruidosos, abundantes) vs. **qubits lógicos** (corregidos, escasos y caros). Shor a escala RSA necesita miles de lógicos.

Slide 36 — El "gap" honesto: lo más grande factorizado de verdad

- En hardware cuántico real, ejecutando Shor: los récords son 15 (2001, IBM, ~8 qubits) y 21 (2012, ~10 qubits). Eso es todo lo confiable.
- Muchos "récords" mediáticos de factorización de números grandes usan circuitos pre-compilados con conocimiento previo de la respuesta, o métodos adiabáticos / de optimización (no Shor) que no escalan. No cuentan como Shor genuino.
- En simulación clásica de Shor (no hardware cuántico) se han factorizado semiprimos de ~12 dígitos (p. ej. $549.755.813.701 = 712.321 \times 771.781$), pero eso mide la simulación, no una ventaja cuántica real.

La distancia entre "15 en hardware real" y "2048 bits en RSA" es enorme. Hay que ser honestos: la amenaza es futura y creíble, no presente.

Slide 37 — Estimaciones de recursos para romper RSA-2048

¿Cuántos qubits harían falta para RSA-2048 con Shor?

- **Gidney & Ekerå (2019):** ~20 millones de qubits físicos ruidosos, ~8 horas.
- **Gidney (mayo 2025, arXiv:2505.15917):** **menos de 1 millón** de qubits físicos ruidosos, **< 1 semana** — unos **1399 qubits lógicos** codificados. Una reducción de ~20× respecto de 2019, vía mejor aritmética modular y codificación más eficiente.

Lectura:

- La barrera **sigue siendo enorme** frente a las máquinas actuales (cientos de qubits físicos, ~decenas de lógicos en el mejor caso).
- Pero las estimaciones **bajan año a año**: el blanco se acerca por mejoras de algoritmo y de corrección de errores, no solo por más hardware.

Slide 38 — Criptografía post-cuántica: NIST PQC

Algoritmos diseñados para resistir a Shor (y a Grover), corribles en hardware clásico actual. **NIST** finalizó los primeros estándares:

Estándar	Algoritmo (competencia)	Nombre final	Tipo	Fecha
FIPS 203	CRYSTALS-Kyber	ML-KEM	Encapsulado de clave (KEM)	13-ago-2024
FIPS 204	CRYSTALS-Dilithium	ML-DSA	Firma digital	13-ago-2024
FIPS 205	SPHINCS+	SLH-DSA	Firma (basada en hash)	13-ago-2024
FIPS 206 (borrador)	FALCON	FN-DSA	Firma	en proceso (2026)
(5.º algoritmo)	HQC	—	KEM (respaldo, código)	selecc. 11-mar-2025; borrador ~2026, final ~2027

- ML-KEM, ML-DSA y FN-DSA se basan en **retículos** (lattices); SLH-DSA en **hashes**; HQC en **códigos** (familia distinta, como respaldo ante un eventual ataque a retículos).

Slide 39 — Qué hacer hoy: "harvest now, decrypt later"

- **Amenaza presente aunque la máquina sea futura:** un adversario puede **capturar hoy** tráfico cifrado y **descifrarlo mañana**, cuando exista la computadora cuántica. Datos con vida útil larga (secretos de estado, salud, propiedad intelectual) ya están en riesgo.
- **Por eso la migración empezó:** estándares finalizados (2024–2025), navegadores y bibliotecas (TLS) ya incorporan **modos híbridos** (clásico + ML-KEM).
- **Recomendación práctica:** inventariar dónde se usa clave pública, priorizar lo de larga vida, y adoptar esquemas **híbridos** durante la transición.

Slide 40 — Conclusiones

- Shor reduce factorizar a hallar el período de $f(x) = k^x \bmod N$; ese período es lo único que corre en la computadora cuántica.
- La cadena algebraica es cerrada: r par $\rightarrow k^r - 1 = (k^{r/2} - 1)(k^{r/2} + 1) \rightarrow$ evitar $k^{r/2} \equiv -1 \rightarrow \gcd(k^{r/2} \pm 1, N)$ da factores, con éxito $\geq 1/2$ por intento.
- La QFT (estimación de fase) extrae el período; es exponencialmente más barata que la FFT en operaciones, pero solo permite **muestrear** la salida.
- Complejidad: **polinómica** (Shor) vs. **sub-exponencial** (GNFS) — un salto de clase.
- **Hoy** no hay hardware para romper RSA real (récorde reales: 15 y 21), pero las hojas de ruta y las estimaciones decrecientes hacen la amenaza **creíble**.
- La respuesta ya existe: **NIST PQC** (ML-KEM, ML-DSA, SLH-DSA, + HQC) y migración **híbrida**, motivada por "harvest now, decrypt later".

Slide 41 — Referencias

Fuente técnica del algoritmo (estándar):

- M. Nielsen, I. Chuang, *Quantum Computation and Quantum Information*, Cambridge — cap. 5 (order-finding, estimación de fase) y Apéndice A4 (cota de probabilidad, Teorema A4.13).
- P. Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring", FOCS 1994.

Material de la cátedra:

- C. H. Ugwuishiwu et al., "An Overview of Quantum Cryptography and Shor's Algorithm", *IJATCSE* 9(5), 2020 — usado para narrativa/motivación. (No contiene cotas de complejidad ni de probabilidad; tiene una errata en la factorización $(m^{P/2}-1)(m^{P/2}-1)$.)
- Clase 04 — Criptografía: Cifrado asimétrico y firma digital, ITBA (RSA, DH, ElGamal, firmas).

Estado del hardware y estimaciones (consultado junio 2026):

- Google Quantum AI, chip "Willow" (corrección por debajo del umbral), 2024–2025.
- IBM, hoja de ruta a tolerancia a fallos ("Starling" 2029), comunicado del 10-jun-2025.
- C. Gidney, "How to factor 2048-bit RSA integers with less than a million noisy qubits", arXiv:2505.15917, 2025.
- C. Gidney, M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits", 2019.

Criptografía post-cuántica (consultado junio 2026):

- NIST, FIPS 203 (ML-KEM), 204 (ML-DSA), 205 (SLH-DSA), finalizados 13-ago-2024.
- NIST, selección de HQC como 5.º algoritmo (KEM), 11-mar-2025; FIPS 206 (FN-DSA/Falcon) en borrador.

Guía narrativa (no técnica):

- K. Rakhade, "Shor's Factoring Algorithm", Medium (solo orden didáctico).